Claude Code in VS Code

Simplified: start, configure, orchestrate

A first-steps guide for complete beginners. You describe what you want, Claude does the work.

Three parts: start, configure, orchestrate

1 Start

Built-in tools, first steps, the project folder structure

Slides 4-8

2 Configure

CLAUDE.md, the LLM wiki, skills (the big one), and optional extras

Slides 9-20

3 Orchestrate

The full recipe, the Ralph loop, MCP connectors, remote control, habits

Slides 21-31

The vibe: you do almost nothing. Anything you want (skills, rules, agents, settings, connectors), you just describe and ask.

You describe, Claude does the rest

What you do

- Say what you want, in plain words
- Answer Claude's questions
- Read the result and say what to change
- Say "next phase"

What Claude does

- Reads your files and plans the work
- Creates folders, documents, code, decks
- Writes its own skills, CLAUDE.md, rules, agents
- Checks its own work and fixes mistakes

Ask `Create a skill that turns my raw meeting notes into a clean summary.`

Ask `Add to my CLAUDE.md: never send an email without asking me.`

Ask `Install the Gmail connector and summarize my unread emails.`

1

Start

Built-in tools, first steps, folder structure

In this section

Claude's built-in tools

Your first steps

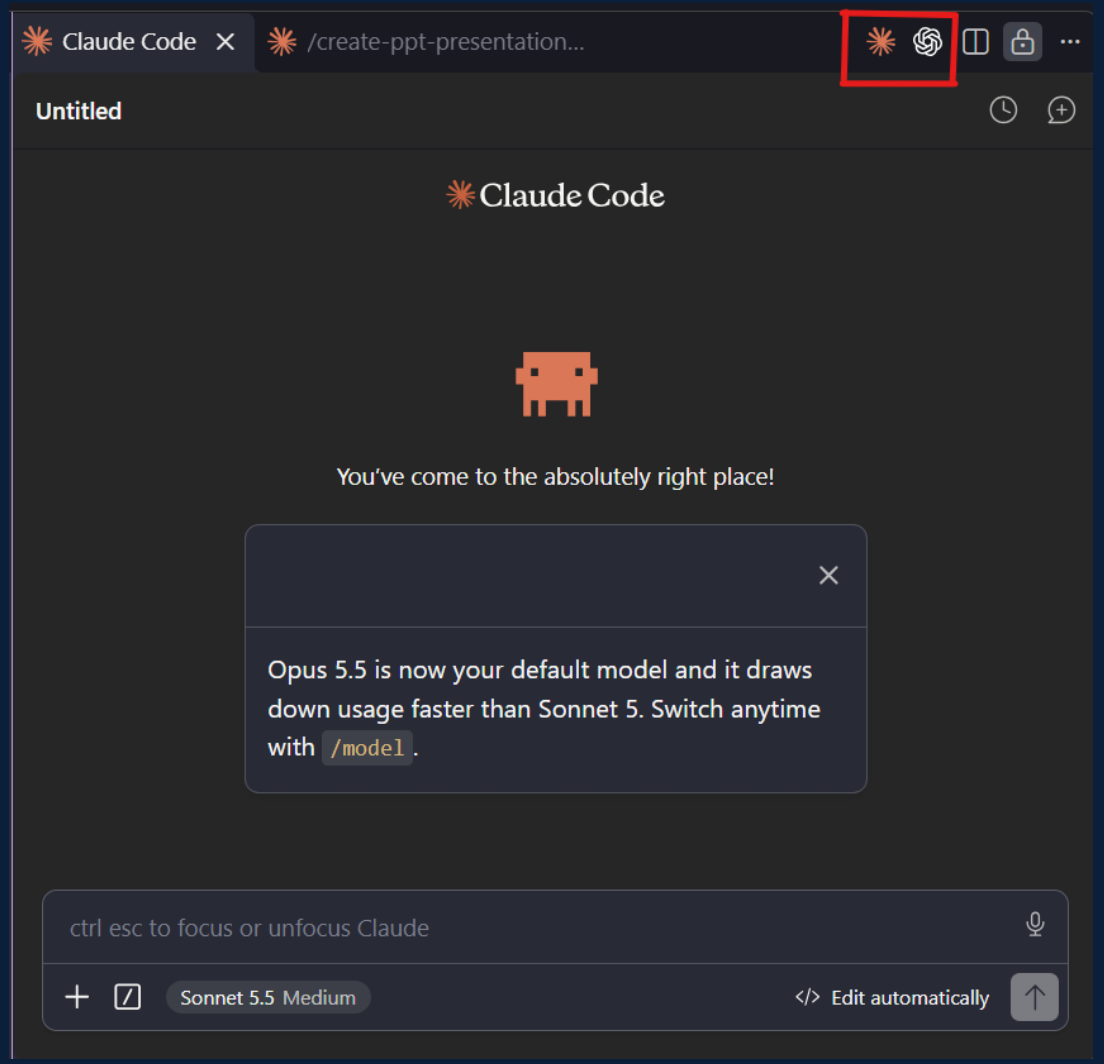
Open the project in VS Code

The recommended folders

Claude works through built-in tools: nothing to install

Read	Opens your files: code, PDFs, Word, Excel, images
Search	Looks for words inside all your files
Find	Finds files by name
Edit	Changes a piece of an existing file
Write	Creates a new file or replaces one
Run	Runs checks and scripts for you
Ask	Asks you to pick between options
Helper	Hands a task to a helper agent

They come with Claude automatically. You see every action in the chat and approve the changes.



The Claude Code extension panel in VS Code

First steps: a folder, a data/ folder, VS Code

1. **Create a project folder**
   One folder per job, e.g. monthly-report. Claude works inside it.

2. **Create a data/ folder inside**
   Put everything in there: briefs, spreadsheets, PDFs, screenshots, notes from colleagues.

3. **Open the folder in VS Code**
   File > Open Folder. Claude can only see what is inside.

4. **Open Claude and describe the task**
   Click the Claude icon, then write what you want in plain words.

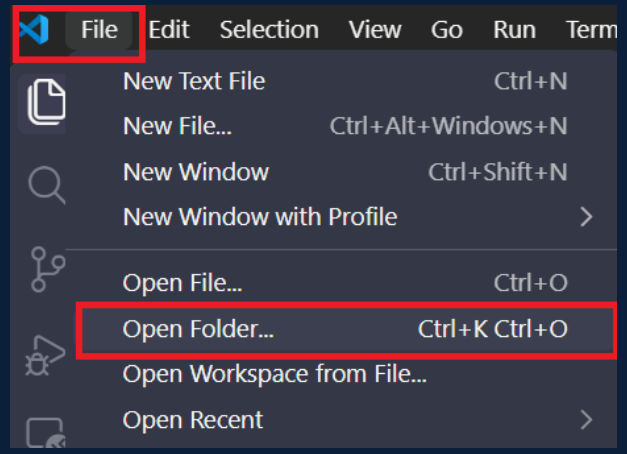
```
monthly-report/
  data/
    client-brief.pdf
    sales-2025.xlsx
    report-template.docx
    notes-from-call.txt
```

Everything Claude needs to know about the job goes in data/. No need to explain it again in chat: Claude reads it.

Any job works: code, reports, research, spreadsheets, emails. A brief and a spreadsheet are as good a start as code.

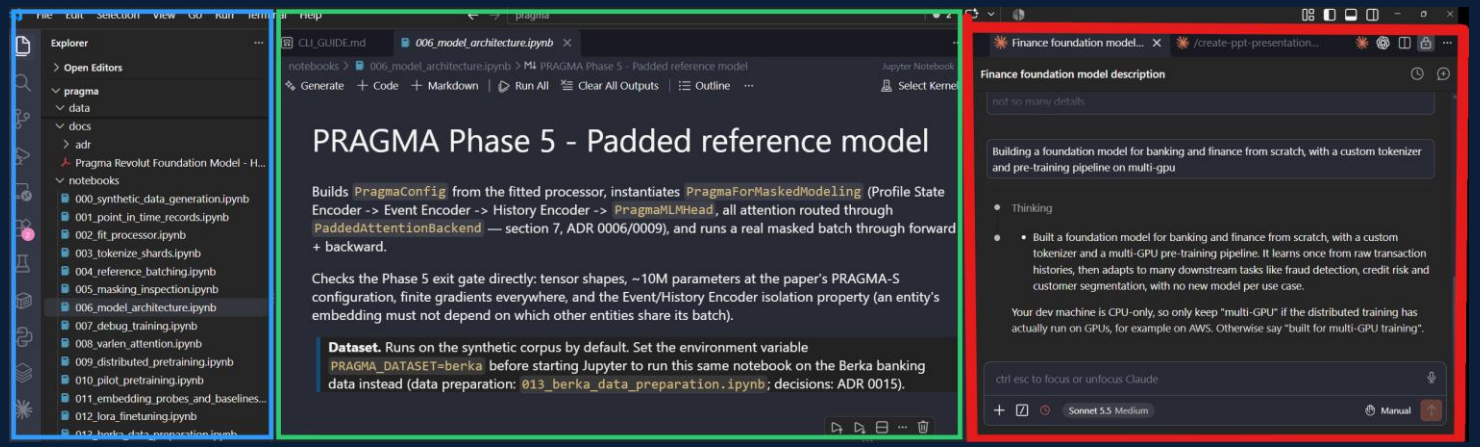
In VS Code: open the folder, open the Claude panel

1 File > Open Folder... and pick your project folder



2 Click the Claude icon: the chat panel opens on the right

3 Describe a task. Review the changes Claude proposes, then accept or push back.



Blue: project files. Green: your work. Red: Claude chat.

The recommended project structure

```
my-project/
  README.md
  CLAUDE.md
  llm-wiki/
    index.md
  plans/
    001-plan-name.md
    progress.md
  data/
  src/
  notebooks/
  outputs/
  test/
```

data/	All inputs: briefs, spreadsheets, PDFs, screenshots, notes
llm-wiki/	Project memory the agents write and read (index.md = the map)
plans/	Numbered plans (001, 002...) and progress.md
src/	Code, when the job needs code
notebooks/	Jupyter notebooks, when you want to explore data
outputs/	Everything Claude produces: reports, decks, files
test/	Checks that prove the code works
CLAUDE.md	Claude's briefing note. README.md = intro for humans

You never build this by hand: the initialize-project skill creates it (slide 17).

2

Configure

Teach Claude your way of working

In this section

What matters most

CLAUDE.md

The LLM wiki

Skills: global and local

Agents, rules, hooks (optional)

What to configure, by importance

Most important

CLAUDE.md

Claude's briefing note, read at the start of every session

Most important

Skills

Reusable playbooks you call with a slash command. The highest value

Very useful

LLM wiki

A project memory the agents write and read

Optional

Agents, rules, hooks, settings

Nice extras. Not necessary to get started

```
.claude/
  skills/
    my-skill/
      SKILL.md
  agents/
  rules/
    reports.md
  settings.json
CLAUDE.md
```

Where it lives: a .claude/ folder in your project (rules go in .claude/rules/), plus CLAUDE.md at the root.

You never write these by hand. Ask Claude: "Create a CLAUDE.md for this project."

CLAUDE.md: Claude's briefing note

What goes in it

- What the project is and how it is organized
- Your instructions: do this, never do that
- Style and tone preferences
- Working habits: ask before big decisions, test small first
- For non-code work: audience, tone, templates, terms to use or avoid

Why it is worth it

- Loaded every session: no re-explaining
- Fewer wrong guesses, fewer corrections
- Global one (~/.claude/CLAUDE.md): your habits in every project
- Project one (in the project root): instructions for that project only
- On conflict, the project one wins

Ask `Create a CLAUDE.md for this project: what it is, how it is organized, and my preferences. (or just type /init)`

Keep it short and concrete. Later: "Add to CLAUDE.md that..."

Inside my CLAUDE.md, section by section

Stack and tooling

Which tools to always use, and the standard folders (data/, plans/, outputs/...)

Style

Keep it simple: plain functions, no over-engineering, brief comments

Planning big work

Numbered plans in plans/, phases, test plan, exit gates, progress.md

Skills

Which skill to use for which job (notebook, deck, video, tests)

Secrets

Keys only in .env. Never print or commit a key

Working style

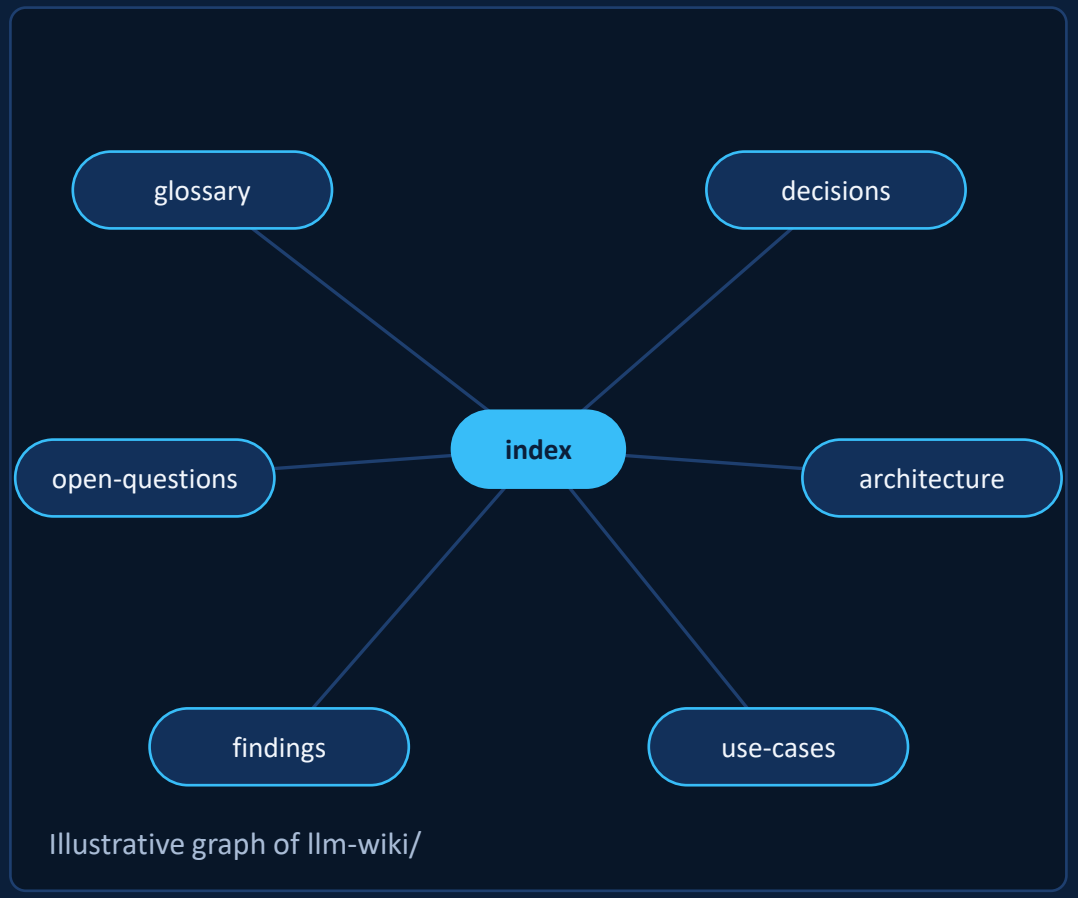
Test small first, report honestly, ask only when blocked, notify me when done

Planning big work and **Working style** are what power the Ralph loop (slide 25).

LLM wiki: a good environment for agents to work in

Why it is worth having

- Agents navigate: index.md is the map of the whole project
- Memory outside the chat: decisions, terms and open questions stay on disk
- Always current: CLAUDE.md tells the agents to update it as they work
- Linked pages: like a small Wikipedia about your project
- Plain text: you can read it too (Obsidian can draw the graph)



Illustrative graph of llm-wiki/

Idea from Andrej Karpathy: a knowledge base that the LLM writes and maintains.

How the wiki is created and kept up to date

```
# index.md
Monthly client report

## Repo map
| Path        | Contains            |
| data/       | all inputs          |
| llm-wiki/   | this wiki           |
| plans/      | plans + progress.md |

## Pages
- [[glossary]]       terms
- [[decisions]]      what and why
- [[open-questions]] unresolved
- [[architecture]]   how it works
```

Step 1: created by a skill

You say "initialize" (slide 17). The initialize-project skill builds llm-wiki/ with index.md, plus the other folders.

Step 2: kept alive by CLAUDE.md

- Read index.md first, every session
- Learn or decide something: update the page
- Add a folder or page: update the repo map
- End of session: plans and progress.md in sync

Pages: glossary, decisions, open questions, architecture, use-cases. Whatever the project needs, linked with [[wikilinks]].

Skills: your reusable playbooks, called with a slash command

```
---
name: create-ppt-presentation
description: Plan and create a
  PowerPoint deck. Use when the user
  asks for a presentation or slides
---
First ask: goal, audience, layout.
Then build the .pptx, then check it.
```

A skill = SKILL.md: a short header (name + when to use) and the instructions.

How to call a skill

```
/create-ppt-presentation 8 slides on Q3 sales
```

```
/initialize-project monthly client report
```

```
/create-notebook explore the sales data
```

Type / in the chat to see all your skills. Or just describe the job: Claude can pick the right skill by itself.

Ask `Create a skill that does X every time I ask for Y.` `Later: Edit that skill so it also Z.`

Non-code skills work the same way: weekly-report, meeting-notes, email-in-my-voice (raw notes in, clean document out).

Global vs local skills: where they live

Global skill: all my projects

```
~/.claude/skills/<name>/SKILL.md
(Windows: C:\Users\you\.claude\skills\)
```

- Available in every project
- Your personal toolbox
- Best for: things you do everywhere (decks, notebooks, project setup)

Local skill: this project only

```
my-project/.claude/skills/<name>/SKILL.md
(inside the project folder)
```

- Available only in this project
- Travels with the project and can be shared with colleagues
- Best for: recipes tied to one project (this client's report format)

Ask `Create a global skill that ...` or `Create a skill just for this project that ...`

CLAUDE.md follows the same split: ~/.claude/CLAUDE.md (global habits) and CLAUDE.md in the project root (local instructions).

My skills in detail: start a project, build a deck

/initialize-project

Starts every project the same way

Aligns first asks the key questions with options and a recommendation, saves decisions in plans/001

Scaffolds README, CLAUDE.md, llm-wiki/ + index.md, plans/, data/, src/, notebooks/, outputs/, test/

Builds small plain functions, nothing "for later", keys only in .env

Tests it runs a small sample before saying done

```
/initialize-project monthly client report
```

/create-ppt-presentation

Plans and builds a PowerPoint deck

Asks first goal, audience, layout and content, and waits for your OK

Clear design dark marine theme, one idea per slide, action titles

Key points pop orange on the 3-4 most important phrases

Checks itself speaker notes, then renders and inspects every slide

```
/create-ppt-presentation 8 slides on Q3 sales
```

Both are global skills (~/.claude/skills/). Claude wrote them; I keep asking it to edit them.

My skills in detail: notebooks and videos

/create-notebook

Builds a Jupyter notebook, local or Kaggle

Picks the target first local (notebooks/, data from data/) or Kaggle

Fixed sections overview, imports, config, load data, run, evaluation plots, conclusion, runtime

Safe by default a debug_mode with a tiny sample; keys only from .env

Tidy names numbered files: 001_explore_data.ipynb

```
/create-notebook explore the sales data
```

/create-video

Turns a script into a narrated video

Input your script or idea

Output an MP4 with voiceover and subtitles

Where works in the project root and saves to outputs/

Calls its helpers uses other installed video skills itself; you just call this one

```
/create-video explainer from my script
```

Same pattern every time: ask first, build small, check the result.

My skills in detail: checks that prove the work

/create-tests

Writes light tests for new, important code

Focused only the important new parts of src/, no duplicates

Safe no network, no paid APIs, no real keys or data

Light a handful of tests per part; adds the tools if missing

Honest if a test finds a real bug, it reports it; it does not bend the test

```
/create-tests
```

/run-tests

Runs the quality gate and reports

Runs three checks pytest (tests), ruff (style), mypy (types)

Same as CI the exact checks an automated pipeline would run

Honest report counts, failing tests, key errors, likely cause

Hands off does not edit code or tests unless you ask; logs results in progress.md

```
/run-tests
```

These two give the loop its pass-or-fail checks (slide 27). I call them only when I ask.

Agents, rules, hooks, settings: optional extras

Agents

Specialist helpers with their own instructions. Example: a reviewer that checks the work and never edits.

```
Create a reviewer agent that checks each phase.
```

Rules

Markdown files in .claude/rules/, one topic per file. Load every session like CLAUDE.md, or only for certain files.

```
Create a rule file: reports always use a formal tone.
```

Hooks

Run something automatically on an event, e.g. a notification when Claude finishes.

```
Add a hook that notifies me when you finish.
```

Settings

Permissions: which actions Claude may do without asking.

```
Allow reading files without asking.
```

Not necessary. Skills and CLAUDE.md are higher value. Start there. Add these only when you feel the need, and just ask Claude to set them up.

3

Orchestrate

Let Claude run the whole job

In this section

The full recipe, step by step

Talk it through

The Ralph loop, hands-on

Plans, progress.md, exit gates

Connectors (MCP), remote, habits

The full recipe, from empty folder to done

1

Create the project folder

One folder per job

2

Create data/ and put everything in it

Briefs, sheets, PDFs, notes

3

Open it in VS Code + Claude panel

File > Open Folder

4

Talk it through

Describe, brainstorm, let Claude question you

5

/initialize-project

Builds CLAUDE.md, wiki, folders, plan

6

Ask for a detailed plan

Phases, test plan, exit gates

7

Run phase 1

Claude builds, checks, fixes, logs

8

"next phase"

Repeat until done (the Ralph loop)

Steps 1-3 are the first steps (slides 6-7). The next slides cover steps 4-8.

Step 4: talk it through before anything is built

What to do

- Describe the problem and who it is for
- Brainstorm ideas and options together
- Let Claude ask you questions until doubts are gone
- Decide, out loud. Claude writes it down later

```
> Read data/. The problem: my monthly
  client report takes two days.
  Don't plan or build yet.
  Ask me what is unclear and give me
  options, with a recommendation.
```

No talk needed? Drop the material in data/ and go straight to /initialize-project.

Your only job here is to answer questions and choose. Claude does the thinking work.

The Ralph loop: keep looping until the checks pass



Why it works

- Each pass starts clean
- Memory lives in files, not in the chat
- Claude finds and fixes its own mistakes
- Works for reports, research and data too

Next: how to set it up, hands-on. It takes three asks.

Set up the loop: three asks, nothing to write by hand

1 Instructions in CLAUDE.md

Once, in your global CLAUDE.md. Loaded every session.

```
Add to my CLAUDE.md:
- Big work: numbered plan
  in plans/, with phases,
  test plan, exit gates
- Keep progress.md current:
  status, evidence, decisions,
  known issues
- Test on a small sample first
- Ask only when blocked
```

2 A skill sets up the project

Once per project. It creates everything.

```
/initialize-project
Use our conversation and what
is in data/.

It builds:
CLAUDE.md, llm-wiki/, plans/,
progress.md, folders
```

3 A detailed plan with gates

Once per big job.

```
Make a detailed plan in
phases,
with a test plan and an exit
gate for each phase.

Then:
> Do phase 1
> next phase
```

After this, the loop is automatic: CLAUDE.md is read every session, so "next phase" is enough.

Two files drive the loop: the plan and progress.md

```
plans/001-plan-monthly-report.md

## Phase 1: load and clean data
- Clean the sales sheet
- Exit gate: row count = source

## Phase 2: build the Word report
- Fill the template from data/
- Exit gate: all 6 fields filled,
  numbers match the sheet

## Phase 3: review and export
- Exit gate: no TODO text left
```

```
plans/progress.md

Status: phase 2 of 3, in progress

## Done
- P1 data loaded and cleaned
## In progress
- P2 build the Word report
## Next
- P3 review, export
## Gate results (evidence)
- P1: row count matches (412)
## Decisions
- Use the template in data/
## Known issues
- 3 rows miss Client ID
```

Numbered plans: never rewrite an old one. progress.md is the memory: a new chat or a new day, Claude re-reads it and continues.

Exit gates: a pass-or-fail check after every phase

Code

- /run-tests passes (tests, style, types)
- A small sample run works end to end
- Notebook runs top to bottom in debug mode

Documents, data, anything else

- Word doc: client name and date are filled in
- Spreadsheet totals match the source file
- Every slide has a title and speaker notes
- No TODO or placeholder text left

Gate fails Claude reads the error, fixes it, logs it under Known issues, runs the gate again

Gate passes Claude writes the evidence in progress.md, marks the phase done, moves on

Ask `Write a script that checks the report against the brief, and use it as the gate.`

Run it: one phase at a time, or all at once

Careful: one phase, then "next phase"

- You read the evidence after every phase
- Best for new, risky or paid work
- Easy to steer between phases

```
> next phase
```

Autopilot: all phases in one go

- Claude loops phase by phase, gate by gate
- Best for clear plans with solid gates
- Plugin /ralph-loop and /goal automate it

```
> Do all the phases in the plan. Stop only
  if a gate fails twice or you need a decision.
```

Start careful. Move to autopilot once the plan and the gates earn your trust.

Connect outside tools: Gmail, Drive and more

Google Drive

Read and write your files in Drive

Gmail and Calendar

Read, summarize and draft emails; check your agenda

Slack, Notion, GitHub...

Chat, notes and code hosting

Ask `Install the Gmail connector for me, then summarize my unread emails.`

- It is called MCP: a standard way to plug outside tools into Claude
- Add it once, use it in any chat
- Only add connectors you trust

Take it with you, and keep learning inside the tool

/remote-control

Continue a running session from another device. Start a long task on your computer, then check on it from your phone.

/powerup

Interactive in-product lessons for features you have not tried yet. A good way to find new tricks.

Type / in the chat to see every command and skill you have.

Eight habits to take home

1 Folder + data/

One folder per job, everything in data/

2 Initialize

/initialize-project builds the structure

3 CLAUDE.md

Your instructions, loaded every session

4 LLM wiki

Project memory the agents keep updated

5 Skills

Turn what you repeat into a slash command

6 Plan + gates

Phases, each with a pass-or-fail check

7 progress.md

The memory: fresh chat, say "next phase"

8 Just ask

Skills, rules, hooks, connectors: Claude writes them

You describe. Claude does the rest.